ARE SELF-EVOLVING AGENT SKILLS LEARNING SKILLS?

Anonymous Submission

ABSTRACT

Higher post-evolution scores raise an identification question: what improvement is carried by the persistent skill, and what remains useful after the task changes? We examine this question through a diagnosis of SkillEvolver, SPARK, and CoEvoSkills, connecting their construction procedures, completed artifacts, and downstream behavior. Repeated paired evaluations establish source-task utility, controlled variations probe its scope, and skill contents and execution traces explain contrasting reuse outcomes. The evidence separates recurring benchmark gains from reusable decision guidance: positive source-task benefits coexist with sharply different responses to changed instances and requirements. Successful cases preserve criteria for selecting valid results and maintaining task constraints. This analysis connects observed gains to what the skill helps an executor decide, making behavioral persistence and attribution central to evaluating skill learning.

***Index Terms*—** Agent skills, self-evolution, diagnostic evaluation, skill reuse, generalization

1. INTRODUCTION

Agent skills externalize behavior into persistent instructions, scripts, and references that can be inspected and reused without changing model parameters. SkillEvolver, SPARK, and CoEvoSkills construct these artifacts from execution experience through deployment-grounded revision, verified-trajectory distillation, and co-evolutionary verification [1–3]. A higher score after this process raises the question in our title: what has the system learned as a skill?

The final score is produced through several steps. Construction determines which guidance enters the artifact; the executor decides how to use that guidance; and a verifier assesses the resulting output. A skill can preserve a useful decision rule, an instance-specific choice, or instructions that a later executor applies incompletely. These possibilities can yield similar source-task scores while producing different behavior when the input or requirement changes. Locating the improvement therefore requires examining the persistent artifact together with its deployment.

We pursue two connected research objectives. First, we examine whether completed skills provide recurring utility and how far that utility extends under controlled task variation. Second, we investigate which decisions preserved in the skills help explain successful and failed reuse, and how those decisions relate to the feedback used during construction. This keeps the existence and scope of a benefit separate from the explanation of how it arises.

Our empirical diagnosis begins with two standard-task runs per method, pairing each skill condition with a no-skill execution of the same task. We then apply frozen skills to changes in input representation, task instances, and coupled requirements. Finally, contrasting signal-detection and scheduling cases connect the observed outcomes to the guidance used by the executor. The sequence links the score, the conditions under which it persists, and the decisions carried by the artifact.

The central finding is a separation between recurring source-task benefit and reusable behavior. All three methods have positive paired mean gains in both standard-task runs, but this agreement breaks under task variation. SkillEvolver remains positive across all three change types; SPARK and CoEvoSkills lose utility on replacement instances. The cases reveal a corresponding distinction in skill content: finding a plausible signal differs from identifying the required signal, and constructing a feasible schedule differs from preserving the rules of an acceptable repair. Successful reuse accompanies guidance that retains these distinctions.

We contribute an empirical diagnosis of the persistence of completed-skill utility and an analysis of the decision guidance preserved in successful reuse. The repeated outcomes establish utility at deployment; controlled variations reveal its behavioral scope; and artifact and trace evidence connect that scope to reusable decision criteria. Together, these observations address what benchmark gains say about skill learning.

2. PROBLEM FORMULATION

2.1. Skill Revision and Evaluation

At revision round r, an optimizer updates the current persistent skill artifact:

$$s_{r+1} = O(s_r, \tau_r, f_r), \tag{1}$$

where s_r is the current skill, τ_r is an execution trace, and f_r is feedback from execution and evaluation. The operator O denotes the method's complete update pathway, including its native merge, validation, selection, or adoption mechanism. A generated candidate that is rejected or never adopted is not s_{r+1}.

The agent then executes task x with the frozen skill:

$$o_r = A(x, s_r, z_r), \tag{2}$$

where z_r represents model sampling, tool use, and environment state, and o_r is the resulting output artifact or state change. A verifier maps the output to a score:

$$y_r = V(x, o_r). \tag{3}$$

自进化智能体技能是否在学习技能?

匿名提交

摘要

进化后的分数提高提出了一个识别问题：哪些改进由持久保存的技能承载，任务变化后哪些指导仍然有用？我们通过诊断SkillEvolver、SPARK 和 CoEvoSkills，将构建过程、最终技能产物和下游行为联系起来。重复的配对评估检验源任务效用，受控变体探查其范围，技能内容与执行轨迹解释不同的复用结果。证据区分了反复出现的基准收益和可复用的决策指导：源任务收益为正，并不意味着技能会同样适应实例和需求变化。成功案例保留了选择有效结果及维持任务约束的标准。这一分析将观察到的收益与技能帮助执行者作出的决策联系起来，以行为的持续性和收益归因为核心评估技能学习。

关键词—智能体技能，自进化，诊断评估，技能复用，泛化

1. 引言

代理技能将行为外化为持久的指令、脚本和引用，这些内容可以在不改变模型参数的情况下被检查和重用。SkillEvolver、SPARK和CoEvoSkills通过部署导向的修订、验证轨迹蒸馏和协同进化验证，从执行经验中构建这些工件 [1–3]。这个过程后的更高分数引发了我们标题中的问题：系统学习到了什么技能?

最终分数是通过几个步骤产生的。构建步骤决定哪些指导进入工件；执行者决定如何使用这些指导；验证器评估最终输出。一个技能可以保留一个有用的决策规则、一个特定实例的选择，或一个后续执行者未完全应用的指令。这些可能性可以在输入或需求变化时产生相似的源任务分数，但行为会不同。因此，定位改进需要检查持久的工件及其部署。

我们追求两个相互关联的研究目标。首先，我们检验完成的技能是否提供持续效用，以及这种效用在受控任务变化下能延伸多远。其次，我们研究技能中保留的哪些决策有助于解释成功和失败的再利用，以及这些决策与构建过程中使用的反馈如何关联。这使收益的存在和范围与收益产生方式的解释分离开来。

我们的实证诊断首先为每种方法开展两次标准任务运行，将技能条件与同题无技能执行配对。随后，将冻结技能应用于输入表示、任务实例和耦合需求的变化。最后，通过信号检测和调度的对照案例，将观察到的结果与执行者使用的指导联系起来。这一顺序连接了分数、收益持续的条件以及技能产物承载的决策。

核心发现是源任务重复收益和可重用行为之间的分离。所有三种方法在标准任务运行中都表现出正向的配对平均增益，但在任务变化时这种一致性被打破。SkillEvolver在所有三种变化类型中都保持正向；SPARK和CoEvoSkills在替换实例上失去效用。案例揭示了相应技能内容上的区别：发现一个合理的信号与识别所需信号不同，构建一个可行的计划与保留可接受修复的规则也不同。成功的重用伴随着保留这些区别的指导。

我们对已完成技能的效用持续性进行实证诊断，并分析成功复用中保留的决策指导。重复结果检验部署时的效用；受控变化揭示其行为范围；技能产物与轨迹证据将这一范围与可复用的决策标准联系起来。这些观察共同说明基准收益为技能学习提供了何种证据。

2. 问题表述

2.1. 技能修订与评估

在修订轮次 r，优化器更新当前持久的技能工件：

$$s_{r+1} = O(s_r, \tau_r, f_r), \tag{1}$$

其中 s_r 是当前技能， τ_r 是执行轨迹，而 f_r 是来自执行和评估的反馈。运算符 O 表示方法完整的更新路径，包括其原生合并、验证、选择或采纳机制。一个被拒绝或从未被采纳的生成候选者不是 s_{r+1}。

代理随后使用冻结的技能执行任务 x：

$$o_r = A(x, s_r, z_r), \tag{2}$$

z_r 表示模型采样、工具使用和环境状态，而 o_r 是结果输出工件或状态变化。验证器将输出映射到分数：

$$y_r = V(x, o_r). \tag{3}$$

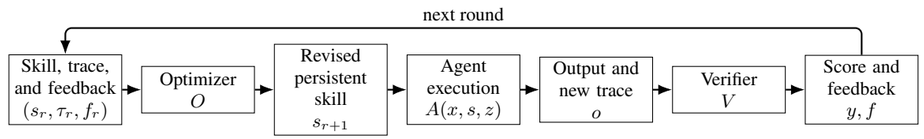


Fig. 1. The skill-evolution lifecycle. Construction produces a persistent artifact; deployment turns its guidance into behavior; verification assigns the observed score. We connect these stages through paired utility, frozen-skill task variation, and inspection of skill use.

2.2. What Does an Observed Gain Identify?

The score $V(x, A(x, s, z))$ depends jointly on the skill, its execution, and the verifier. A skill-versus-no-skill comparison measures the utility of a completed artifact at deployment. A parent-to-final comparison instead measures the change associated with revising an existing artifact. Keeping these contrasts distinct specifies which part of the learning question an evaluation answers.

Our empirical contrast is completed-skill utility. We first examine whether its paired mean is positive in each of two runs on the same task set. We then freeze the source-task skill s_x and apply it to a variant $T_k(x)$, pairing its outcome with a no-skill execution on that variant. The executor can reinterpret inputs and adapt its actions; the skill itself is not revised. Persistence of utility under these changes locates the observed scope of reuse.

Attribution concerns the decisions connecting the artifact to the result. We inspect what the skill tells the executor to retain, recompute, or check, and how that guidance appears in successful and failed executions. Published construction procedures supply an explanation for how such guidance may arise. This separates observed behavior and artifact content from interpretation of the evolution pathway.

3. DOES SKILL UTILITY PERSIST BEYOND THE SOURCE TASK?

We address the existence and scope of completed-skill utility through two contrasts: paired standard-task outcomes across runs, and paired outcomes after changing the task while keeping its source skill frozen.

3.1. Experimental Setup

We evaluate SkillEvolver [1], SPARK [2], and CoEvoSkills [3] on SkillsBench [4], with two runs per method. Each run pairs the method's skill condition with a no-skill execution of the same task. Both runs of each method use the same task set: 81 tasks for SkillEvolver, 84 for SPARK, and 85 for CoEvoSkills.

We report mean verifier reward on a 0–100 scale and the mean within-task difference between the skill and no-skill conditions. For a run with N tasks, the paired gain is

$$\Delta = \frac{100}{N} \sum_{i=1}^{N} \left(r_i^{\text{skill}} - r_i^{\text{no-skill}} \right). \tag{4}$$

We also count tasks with positive, negative, and zero differences.

3.2. Repeated Runs Establish Source-Task Utility

All three methods have positive mean paired gains in both runs (Table 1). SkillEvolver gains 25.61 and 25.02 points, CoEvoSkills gains 10.24 and 9.76, and SPARK gains 1.84 and 6.45. SkillEvolver and CoEvoSkills retain similar gains despite substantial changes in their no-skill scores. The paired comparison reveals recurring utility that absolute skill-condition scores alone would obscure.

The task-level results also differ. SkillEvolver has at least four times as many improvements as regressions in each run; CoEvoSkills improves 17 tasks in each run, with nine and ten regressions. SPARK's first run is nearly balanced, while its second has 13 improvements and five regressions. These results establish recurring source-task utility. We next ask whether the same frozen skills remain useful after the task changes.

Method	Run	N	No skill	Skill	Δ	I/R/T
SkillEvolver	1	81	30.20	55.81	+25.61	27/6/48
	2	81	37.32	62.34	+25.02	26/3/52
SPARK	1	84	17.82	19.66	+1.84	9/8/67
	2	84	14.37	20.81	+6.45	13/5/66
CoEvoSkills	1	85	42.21	52.44	+10.24	17/9/59
	2	85	23.54	33.31	+9.76	17/10/58

Table 1. Two runs per method on SkillsBench. Mean verifier rewards and paired gain Δ use a 0–100 scale. I/R/T counts improved, regressed, and tied tasks.

3.3. Task Variation Tests What Remains Useful

We apply each source-task skill to controlled variants without further evolution. Input-representation changes (P1) alter names, column arrangements, or schemas; task-instance changes (P2) replace the data or instance structure; coupled-requirement changes (P3) alter or add requirements that must be satisfied together. The corresponding decision demands are to remap the input, recompute a solution for a replacement instance, and accommodate changed correctness requirements. A frozen skill must support these adaptations through guidance already present in the artifact.

The complete record contains 222 variants and 886 condition-level results across no skill and the three methods. Table 2 summarizes historical and recovery evaluations for 222 SkillEvolver variants and 221 each for SPARK and CoEvoSkills, pairing each skill condition with the no-skill result on the same variant.

Source-task improvement does not imply useful generalization. All three methods improve mean reward on the source tasks in both runs, but their variant results diverge. SkillEvolver retains a positive mean gain of 11.94 points, SPARK is close to neutral at $+0.65$, and CoEvoSkills falls below the no-skill baseline at -5.01. SkillEvolver also improves more variants than it regresses (47 versus 20), whereas SPARK and CoEvoSkills show the reverse. Thus, agreement on positive source-task utility conceals different responses to changed tasks. Repeated source-task benefit and persistence under variation are distinct observations.

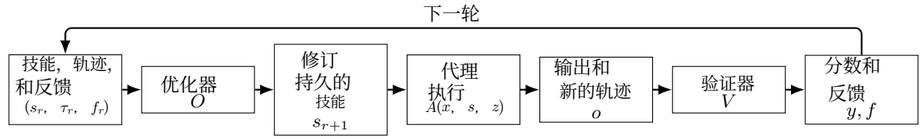


图1。技能进化生命周期。构建产生一个持久的工件；部署将其指导转化为行为；验证分配观察到的分数。我们通过配对效用、冻结技能任务变异以及检查技能使用将这些阶段连接起来。

2.2. 观察到的收益能说明什么?

分数 $V(x, A(x, s, z))$同时取决于技能、其执行以及验证器。技能与无技能的比较衡量部署时完成工件的效用。而父级到最终的比较则衡量修订现有工件相关的变化。保持这些对比的区分指定了评估回答的学习问题的哪个部分。

我们实际采用的对比检验已完成技能的效用。首先，考察同一任务集的两次运行是否均有正向配对平均增益。随后冻结源任务技能，将其用于变体，并与同一变体的无技能执行配对。执行者可以重新理解输入和调整动作，但技能本身不再修订。效用在这些变化下是否持续，界定了观察到的复用范围。

归因关注将工件与结果联系起来的决策。我们检查技能告诉执行者要保留、重新计算或检查的内容，以及这种指导在成功和失败的执行中如何体现。已发布的构建程序为这种指导如何产生提供了解释。这将观察到的行为和工件内容与进化路径的解释分开了。

3. 技能效用是否超出源任务?

我们通过两种对比来探讨完成技能的效用存在性和范围：跨运行的对齐标准任务结果，以及在改变任务时保持其源技能冻结后的对齐结果。

3.1. 实验设置

我们在 SkillsBench [4] 上评估 SkillEvolver [1]、SPARK [2] 和 CoEvoSkills [3]，每种方法运行两次。每次将技能条件与同题无技能执行配对。同一方法的两次运行使用相同任务集：SkillEvolver 81题，SPARK 84题，CoEvoSkills 85题。

我们报告0–100分制的平均验证器奖励，以及技能条件与无技能条件的平均同题差值。对于包含 N 个任务的运行，配对增益为

$$\Delta = \frac{100}{N} \sum_{i=1}^{N} \left(r_i^{\text{skill}} - r_i^{\text{no-skill}} \right). \tag{4}$$

我们还统计了具有正差异、负差异和零差异的任务数量。

3.2. 重复运行验证源任务效用

三种方法在两次运行中的平均配对增益均为正（表1）。SkillEvolver 提高25.61和25.02分，CoEvoSkills 提高10.24和9.76分，SPARK 提高1.84和6.45分。

SkillEvolver 和 CoEvoSkills 的无技能分数虽变化较大，增益却相近。配对比较揭示了反复出现的效用，而技能条件的绝对分数会掩盖这一点。

逐题结果也有差异。SkillEvolver 每次运行的改进数至少为退化数的四倍；CoEvoSkills 两次均改进17题，分别退化9题和10题。SPARK 首次的改进与退化数接近，第二次为13次改进、5次退化。这些结果表明源任务效用反复出现。接下来考察同一冻结技能在任务变化后是否仍然有用。

Method	Run	N	No skill	Skill	Δ	I/R/T
SkillEvolver	1	81	30.20	55.81	+25.61	27/6/48
	2	81	37.32	62.34	+25.02	26/3/52
SPARK	1	84	17.82	19.66	+1.84	9/8/67
	2	84	14.37	20.81	+6.45	13/5/66
CoEvoSkills	1	85	42.21	52.44	+10.24	17/9/59
	2	85	23.54	33.31	+9.76	17/10/58

表1. SkillsBench 上每种方法的两次运行。平均验证器奖励及配对增益 Δ 均按0–100分制报告。I/R/T 分别为改进、退化和持平的任务数。

3.3. 任务变化测试什么仍然有用

我们对每个源任务技能应用于受控变体，不再进行进一步进化。输入表示变化（P1）会改变名称、列排列或模式；任务实例变化（P2）会替换数据或实例结构；耦合需求变化（P3）会改变或添加必须一起满足的需求。相应的决策需求是重新映射输入、为替换实例重新计算解决方案，并适应变化后的正确性需求。一个冻结的技能必须通过工件中已有的指导来支持这些适应性。

完整记录包含222个变体，以及无技能和三种方法共886条条件级结果。表2汇总历史评估与补跑：SkillEvolver 覆盖222个变体，SPARK 和 CoEvoSkills 各覆盖221个。每个技能条件均与同一变体的无技能结果配对。

源任务改进不等于有效泛化。三种方法在两次运行中均提高了源任务平均奖励，但变体结果发生分化。SkillEvolver 保持11.94分的正向平均增益，SPARK 的 +0.65 接近零，CoEvoSkills 则比无技能基线低5.01分。SkillEvolver 的改进数多于退化数（47比20），SPARK 和 CoEvoSkills 则相反。因此，源任务效用均为正，掩盖了技能对任务变化的不同响应。源任务收益反复出现与收益在任务变化后持续，是不同的观察结果。

Replacing the instance produces the largest divergence. SkillEvolver remains positive under all three changes, with its largest gain on P2 (+14.56). P2 is also where SPARK and CoEvoSkills decline most (−5.23 and −9.51). SPARK retains positive averages under representation and requirement changes, but loses its advantage when the underlying instance changes. This separates remapping inputs and adjusting requirements from recomputing choices for a new instance. SkillEvolver's positive gains across P1–P3 show the broadest observed persistence of utility, while CoEvoSkills' negative means show that source-task guidance can become counterproductive after the task changes.

A. Overall utility on task variants					
Method	N	**No skill**	**Skill**	**Δ**	**I/R/T**
SkillEvolver	222	41.87	53.81	+11.94	47/20/155
SPARK	221	41.61	42.25	+0.65	32/37/152
CoEvoSkills	221	41.61	36.60	−5.01	26/41/154

B. Utility by task change: (N) Δ			
Method	**P1**	**P2**	**P3**
SkillEvolver	(77) +11.85	(68) +14.56	(77) +9.72
SPARK	(77) +2.00	(68) −5.23	(76) +4.53
CoEvoSkills	(77) −3.59	(68) −9.51	(76) −2.42

Table 2. Skill utility under task variation: descriptive summary of historical and recovery evaluations. Each method is paired with its same-variant no-skill baseline. P1/P2/P3 denote representation, instance, and requirement changes. Panel B gives variant count N and paired gain Δ; scores and I/R/T follow Table 1.

4. WHAT IN THE SKILL EXPLAINS ITS BEHAVIOR?

The second research objective is to connect the observed outcome to the persistent guidance used during execution. Signal detection and scheduling expose two distinct decisions: selecting the valid candidate and preserving the conditions of a correct repair. We inspect both the rules written into the skills and the behavior observed when they are reused.

4.1. Distinguishing a Candidate from a Valid Answer

In exoplanet detection, the SkillEvolver skill supports successful detection after input columns are rearranged (P1) and after the task requires two signals (P3). The CoEvoSkills skill fails both variants: its use leads to stellar modulation being reported as the target signal, and a signal and its harmonic being treated as two separate signals. Both skills fail the replacement-instance P2 variant.

The difference is visible in what the skills ask the executor to check. SkillEvolver separates candidate discovery, signal diagnosis, and refinement. Its guidance addresses misleading peaks, transit-like durations, and harmonic relationships. CoEvoSkills also includes validation, including phase-fold, odd/even, and residual checks, but its use in these variants does not resolve the misleading candidates. The presence of a checking routine is therefore less informative than whether its criteria distinguish the alternatives that matter on the changed task. SkillEvolver's P1 and P3 successes show this guidance being useful under changed inputs and requirements. Its P2 failure locates a different boundary within the same task family: the retained criteria do not resolve that replacement instance, despite a positive P2 average across the broader evaluation.

4.2. Preserving the Rules That Define a Correct Repair

In scheduling, the SkillEvolver and SPARK skills succeed on all three variants; the CoEvoSkills skill succeeds only on P3. In P1, using CoEvoSkills yields an otherwise feasible schedule that fails a required local-minimality check on the baseline-relative repair. Feasibility checks alone do not capture this local-minimality requirement.

The SkillEvolver skill explicitly retains local minimality and baseline-relative repair rules. These rules specify how an acceptable repair differs from an arbitrary feasible schedule, and remain relevant after the input names or scheduling instance change. The CoEvoSkills skill covers several structural and feasibility checks, yet the failed deployment misses the decisive repair condition. The contrast concerns whether the decisive repair condition is preserved in guidance and enforced during its use. SPARK's success across the same three variants also shows that this form of successful reuse is not exclusive to one method.

4.3. Construction Feedback and Skill Content

The construction procedures help interpret how these decision criteria enter a skill. SkillEvolver compares successful and failed deployments of a candidate skill and patches missing or misleading guidance. Its independent audit also checks reliance on training-instance constants [1]. This exposes failures in using the skill and encourages rules that a later executor can apply to a different input. The scheduling and signal-detection skills reflect that emphasis by retaining criteria for checking a repair or discriminating between candidate signals.

SPARK grounds skill distillation in environment-verified trajectories, preserving execution evidence, checks, and lessons [2]. Its scheduling result shows that such evidence can support successful reuse. Its mixed aggregate profile raises a separate question about abstraction: a verified execution supplies evidence for what worked on the source task, while changed instances require determining which choices should be retained and which should be recomputed. The observed decline on P2 makes that distinction especially relevant to interpreting its results.

CoEvoSkills obtains detailed revision feedback from a co-evolving surrogate verifier, while the ground-truth oracle returns an opaque pass/fail signal [3]. This makes the surrogate's coverage of correctness conditions relevant to the guidance retained in the skill. This provides a plausible account of the scheduling failure: the skill preserves many checks, but successful reuse still depends on a condition those checks did not enforce in that deployment. Across the cases, feedback coverage and abstraction from the source instance are candidate explanations for the different scopes of utility.

5. INTERPRETING GAINS AS EVIDENCE OF SKILL LEARNING

5.1. Utility, Scope, and Attribution Answer Different Questions

Positive paired gains in both runs establish recurring source-task utility of the completed skills. Task variation asks a different question: which parts of that usefulness survive when the original input or solution no longer suffices? The three methods agree on the first observation and diverge on the second. This is the diagnostic result behind the performance differences. A source-task average describes a benefit in one setting; the variation profile reveals what demands the same artifact can accommodate.

实例替换带来的分化最明显。SkillEvolver 在三类变化下均为正，P2 增益最大（+14.56）。SPARK 和 CoEvoSkills 在 P2 上的下降也最明显（−5.23 和 −9.51）。SPARK 在表示和需求变化下平均增益为正，底层实例变化后却失去优势。这区分了重新映射输入、适应需求和为新实例重新计算决策。SkillEvolver 在 P1–P3 上均有正向增益，表现出最广的效用持续范围；CoEvoSkills 的负向均值则说明，源任务指导在任务变化后可能适得其反。

A. Overall utility on task variants					
Method	N	**No skill**	**Skill**	**Δ**	**I/R/T**
SkillEvolver	222	41.87	53.81	+11.94	47/20/155
SPARK	221	41.61	42.25	+0.65	32/37/152
CoEvoSkills	221	41.61	36.60	−5.01	26/41/154

B. Utility by task change: (N) Δ			
Method	**P1**	**P2**	**P3**
SkillEvolver	(77) +11.85	(68) +14.56	(77) +9.72
SPARK	(77) +2.00	(68) −5.23	(76) +4.53
CoEvoSkills	(77) −3.59	(68) −9.51	(76) −2.42

表2. 任务变化下的技能效用，描述性汇总历史评估与补跑结果。每种方法均与同一变体的无技能基线配对。P1/P2/P3 分别为表示、实例和需求变化。B部分给出变体数 N 与配对增益 Δ；分数及 I/R/T 定义同表1。

4. 技能中什么解释其行为?

第二个研究目标是将观察到的结果与执行过程中使用的持久性指导联系起来。信号检测和调度暴露了两个不同的决策：选择有效的候选和保持正确修复的条件。我们检查了写入技能的规则以及它们被重用时观察到的行为。

4.1. 区分候选与有效答案

在系外行星探测中，SkillEvolver技能在输入列重新排列（P1）后和任务要求两个信号（P3）后支持成功探测。CoEvoSkills技能两种变体都失败：其使用导致报告恒星调制为目标信号，并将一个信号及其谐波视为两个独立信号。两种技能都失败替换实例P2变体。
差异体现在技能要求执行者检查什么。SkillEvolver 将候选发现、信号诊断和精炼分开，指导执行者处理误导峰值、符合凌日的持续时间及谐波关系。CoEvoSkills 也包含相位折叠、奇偶检查和残差检查，但在这些变体的执行中未能排除误导候选。因此，存在检查流程不如其标准能否区分关键候选更有解释力。SkillEvolver 在 P1 和 P3 成功，说明这一指导适用于变化后的输入和需求。其 P2 失败则揭示同一任务系列中的另一处边界：虽然更广泛评估的 P2 均值为正，保留的标准未能解决这个替换实例。

4.2. 保持定义正确修复的规则

在调度方面，SkillEvolver和SPARK技能在所有三种变体中都成功；CoEvoSkills技能仅在P3中成功。在P1中，使用CoEvoSkills会产生一个否则可行的调度方案，但在基线相对修复中会失败所需的局部最小性检查。仅进行可行性检查并不能捕捉这种局部最小性要求。

SkillEvolver技能明确保留局部最小性和基线相对修复规则。这些规则指定可接受修复与任意可行调度有何不同，并在输入名称或调度实例更改后仍然相关。CoEvoSkills技能涵盖几种结构和可行性检查，但失败的部署错过了决定性修复条件。对比在于决定性修复条件是否在指导中保持并在使用时强制执行。SPARK在相同三种变体中的成功也表明这种成功重用并非专属于一种方法。

4.3. 构造反馈与技能内容

构造流程有助于解释这些决策标准如何进入技能。
SkillEvolver 比较候选技能的成功和失败部署，并修补缺失或误导性的指导。其独立的审计还会检查对训练实例常量 [1] 的依赖。这暴露了使用技能的失败，并鼓励后续执行者可以应用于不同输入的规则。调度和信号检测技能通过保留检查修复或区分候选信号的标准，反映了这种侧重。

SPARK 从经环境验证的轨迹中蒸馏技能，保留执行证据、检查和经验教训 [2]。其调度结果表明这些证据能够支持成功复用。总体结果有升有降，进一步提出了抽象问题：验证过的执行说明哪些做法在源任务上有效；实例变化后，仍需决定哪些选择应保留、哪些应重新计算。P2 上的下降使这一区分对理解其结果尤为重要。

CoEvoSkills 从协同进化的替代验证器获取详细修订反馈，而真实评测器只返回不透明的通过或失败信号 [3]。因此，替代验证器对正确性条件的覆盖范围，与技能保留的指导相关。这为调度失败提供了一个可能的解释：技能虽保留多个检查，成功复用仍依赖此次执行中未落实的条件。在这些案例中，反馈覆盖范围和对源实例的抽象程度，是效用范围不同的候选解释。

5. 解释收益作为技能学习的证据

5.1. 效用、范围和归因 回答不同的问题

两次运行中的正向配对增益表明，已完成技能的源任务效用反复出现。任务变化回答另一个问题：当原始输入或解法不再适用时，哪些效用仍然保留？三种方法在第一项观察上一致，在第二项上分化。这是产生性能差异的诊断发现。源任务均值描述特定环境下的收益；变体结果则揭示同一技能产物能够满足哪些要求。

Average gain and improvement frequency also reveal different behavior. SPARK's slightly positive variant mean combines more regressions than improvements, so near-zero utility masks opposing changes. SkillEvolver combines a positive mean with more improvements, whereas CoEvoSkills combines a negative mean with more regressions. Pairing these summaries with the cases makes the interpretation concrete: utility depends on whether the guidance helps the executor make the changed task's decisive choices.

5.2. What the Persistent Artifact Must Preserve

The cases suggest that reusable skill learning preserves the relationship between a procedure and the conditions that make its result valid. In signal detection, this relationship distinguishes a target from misleading candidates. In scheduling, it connects repair actions to baseline-relative constraints. A procedure can remain executable while this relationship is lost. Conversely, a skill can guide a new solution by preserving the checks and adaptation rules that still apply.

This connects the diagnosis back to the evolution lifecycle. Feedback supplies evidence about failures; the skill expresses a rule extracted from that evidence; the executor must use it in the new setting; and the verifier records the outcome. The observed cases expose whether the guidance survives this handoff. Construction, adoption, and useful deployment are separate events, even when they contribute to the same reported score.

Evaluation should therefore follow the artifact from its source-task contribution to its changed-task behavior. Repeated paired outcomes show whether utility recurs, frozen variants test which decisions remain applicable, and skill/trace inspection explains where reuse succeeds or breaks. When the claim concerns the improvement produced by revision itself, the corresponding contrast is the actual parent and adopted final artifact. The research target is a persistent change in useful behavior, with the score serving as one observation of that change.

6. RELATED WORK

Learning persistent skills from experience. SkillEvolver, SPARK, and CoEvoSkills make experience persistent through different construction loops: deployment-grounded revision, verified-trajectory distillation, and co-evolutionary verification, respectively [1–3]. These methods motivate our focus on the artifact delivered to a later executor. We study how that artifact's usefulness changes after its source task is varied, and interpret the resulting behavior through its retained guidance.

Evaluating skill utility and use. SkillsBench evaluates skill utility across diverse tasks, while SWE-Skills-Bench studies skill use in software engineering [4, 5]. SkillLearnBench evaluates continual skill generation on real-world tasks [6]. From Raw Experience to Skill Consumption separates experience generation, skill extraction, and downstream consumption [7]. Our study connects these evaluation levels to the identification of skill learning: repeated outcomes characterize completed-skill utility, task changes test its persistence, and skill-level examples explain the decisions behind contrasting outcomes. The unit of interpretation is the persistent artifact in use.

7. CONCLUSION

A higher score becomes informative about skill learning when it is connected to what the persistent artifact helps an executor do. Our repeated evaluations show positive source-task utility for all three methods, while frozen task variations separate the conditions under which that utility persists. SkillEvolver retains positive gains across the three change types; SPARK and CoEvoSkills expose narrower or adverse reuse patterns. The contrasting cases connect successful reuse to criteria for identifying valid answers and preserving repair constraints. These findings distinguish useful completed artifacts, their behavioral scope, and explanations grounded in skill content. They return the learning question to persistent decision guidance: what is retained from experience, when does it remain useful, and how does it shape the next execution?

平均增益和改进频率也揭示了不同的行为。SPARK 的变体均值略为正向，退化数却多于改进数，因此接近零的效用掩盖了方向相反的变化。SkillEvolver 同时具有正向均值和更多改进，CoEvoSkills 则同时具有负向均值和更多退化。结合案例，这些结果得到更具体的解释：效用取决于技能指导是否帮助执行者作出变化任务中的关键决策。

5.2. 持久化工件必须保留的内容

案例表明，可重用技能学习保留了程序与其结果有效条件之间的关系。在信号检测中，这种关系区分了目标与误导性候选。在调度中，它将修复操作与基线相对约束相连接。当这种关系丢失时，程序仍可保持可执行性。相反，技能可以通过保留仍然适用的检查和自适应规则来指导新解决方案。

这使诊断与进化生命周期重新关联。反馈提供了关于失败的证据；技能表达了从该证据中提取的规则；执行器必须在新的环境中使用它；验证器记录了结果。观察到的案例揭示了指导是否在这种交接中得以保留。构建、采用和有效部署是分开的事件，即使它们对同一报告分数有贡献。

因此，评估应从技能产物对源任务的贡献出发，追踪任务变化后的行为。重复的配对结果检验效用是否反复出现；冻结技能的变体评估检验哪些决策仍适用；技能和轨迹检查解释复用成功或失败的环节。若研究主张涉及修订本身带来的改进，对应比较应为实际的父技能与最终采用的技能。研究对象是持久且有用的行为变化，分数是对这一变化的一种观察。

6. 相关工作

从经验中学习持久技能。 SkillEvolver、SPARK和CoEvoSkills通过不同的构建循环使经验持久化：基于部署的修订、验证轨迹的蒸馏和协同进化的验证，分别 [1–3]。这些方法促使我们关注交付给后续执行者的工件。我们研究该工件在源任务变化后的有用性如何变化，并通过其保留的指导来解释其行为。

评估技能效用与使用。SkillsBench 覆盖多样任务，SWE-Skills-Bench 研究软件工程中的技能使用 [4,5]。SkillLearnBench 评估真实任务上的持续技能生成 [6]。《从原始经验到技能使用》区分经验生成、技能提取与下游使用 [7]。我们将这些评估层面与技能学习的识别联系起来：重复结果刻画已完成技能的效用，任务变化检验其持续性，技能层面的案例解释造成不同结果的决策。解释的对象是执行中使用的持久技能产物。

7. 结论

当更高的分数与持久的工件帮助执行者完成的事情相关联时，它就变成了关于技能学习的有用信息。我们的重复评估显示，所有三种方法都具有积极的源任务效用，而冻结的任务变化则区分了这种效用持续的条件。SkillEvolver在三种变化类型中都保持了正向收益；SPARK和CoEvoSkills则暴露出更狭窄或不利的使用模式。这些对比案例将成功的重用与识别有效答案和保持修复约束的标准联系起来。这些发现区分了有用的已完成工件、它们的行为范围，以及基于技能内容的解释。它们将学习问题回归到持久的决策指导：从经验中保留什么，何时它仍然有用，以及它如何塑造下一次执行?

8. REFERENCES

[1] Genrui Zhang, Erle Zhu, Jinfeng Zhou, Caiyan Jia, and Hongning Wang, "SkillEvolver: Skill Learning as a Meta-Skill," 2026.

[2] Yang Zhou, Zihan Dong, Zhenting Wang, Can Jin, Shiyu Zhao, Bangwei Guo, Difei Gu, Linjun Zhang, Mu Zhou, and Dimitris N. Metaxas, "Evidence Over Plans: Online Trajectory Verification for Skill Distillation," 2026.

[3] Hanrong Zhang, Shicheng Fan, Henry Peng Zou, Yankai Chen, Zhenting Wang, Jiayu Zhou, Chengze Li, Wei-Chieh Huang, Yifei Yao, Kening Zheng, Xue Liu, Xiaoxiao Li, and Philip S. Yu, "CoEvoSkills: Self-Evolving Agent Skills via Co-Evolutionary Verification," 2026.

[4] Xiangyi Li, Yimin Liu, Wenbo Chen, Bingran You, Zonglin Di, Yifeng He, Shenghan Zheng, Kyoung Whan Choe, Jiankai Sun, Shuyi Wang, Chujun Tao, Binxu Li, Xuandong Zhao, Hejia Geng, Xiaojun Wu, Junwei Zhou, Xiaokun Chen, Hanwen Xing, Yubo Li, Qunhong Zeng, Di Wang, Yuanli Wang, Roey Ben Chaim, Penghao Jiang, Haotian Shen, Luyang Kong, Xinyi Liu, Runhui Wang, Xuanqing Liu, Jiachen Li, Xin Lan, Yueqian Lin, Wengao Ye, Junwei He, Songlin Li, Yue Zhang, Yipeng Gao, Yijiang Li, Ze Ma, Liqiang Jing, Tianyu Wang, Kaixin Li, Yiqi Xue, Haoran Lyu, Yizhuo He, Yuchen Tian, Shutong Wu, Bowei Wang, Yixuan Gao, Bo Chen, Litong Liu, Sikai Cheng, Jiajun Bao, Shuaicheng Tong, Shuwen Xu, Terry Yue Zhuo, Tinghan Ye, Qi Qi, Miao Li, Longtai Liao, Zelin Tan, Chang Shi, Xilin Tang, Srinath Tankasala, Boqin Yuan, Yaoyao Qian, Jianhong Tu, Chenguang Wang, Yizhou Sun, Wei Wang, Aaron Taylor, Ziyue Yang, Changkun Guan, Zhikang Dong, Xinyu Zhang, Steven Dillmann, Han chung Lee, and Dawn Song, "Skills-Bench: Benchmarking How Well Agent Skills Work Across Diverse Tasks," 2026.

[5] Tingxu Han, Yi Zhang, Wei Song, Chunrong Fang, Zhenyu Chen, Youcheng Sun, and Lijie Hu, "SWE-Skills-Bench: Do Agent Skills Actually Help in Real-World Software Engineering?," 2026.

[6] Shanshan Zhong, Yi Lu, Jingjie Ning, Yibing Wan, Lihan Feng, Yuyi Ao, Leonardo F. R. Ribeiro, Markus Dreyer, Sean Ammirati, and Chenyan Xiong, "SkillLearnBench: Benchmarking Continual Learning Methods for Agent Skill Generation on Real-World Tasks," 2026.

[7] Zisu Huang, Jingwen Xu, Yifan Yang, Ziyang Gong, Qihao Yang, Muzhao Tian, Xiaohua Wang, Changze Lv, Xuemei Gao, Qi Dai, Bei Liu, Kai Qiu, Xue Yang, Dongdong Chen, Xiaoqing Zheng, and Chong Luo, "From Raw Experience to Skill Consumption: A Systematic Study of Model-Generated Agent Skills," 2026.

8. REFERENCES

[1] Genrui Zhang, Erle Zhu, Jinfeng Zhou, Caiyan Jia, and Hongning Wang, "SkillEvolver: Skill Learning as a Meta-Skill," 2026.

[2] Yang Zhou, Zihan Dong, Zhenting Wang, Can Jin, Shiyu Zhao, Bangwei Guo, Difei Gu, Linjun Zhang, Mu Zhou, and Dimitris N. Metaxas, "Evidence Over Plans: Online Trajectory Verification for Skill Distillation," 2026.

[3] Hanrong Zhang, Shicheng Fan, Henry Peng Zou, Yankai Chen, Zhenting Wang, Jiayu Zhou, Chengze Li, Wei-Chieh Huang, Yifei Yao, Kening Zheng, Xue Liu, Xiaoxiao Li, and Philip S. Yu, "CoEvoSkills: Self-Evolving Agent Skills via Co-Evolutionary Verification," 2026.

[4] Xiangyi Li, Yimin Liu, Wenbo Chen, Bingran You, Zonglin Di, Yifeng He, Shenghan Zheng, Kyoung Whan Choe, Jiankai Sun, Shuyi Wang, Chujun Tao, Binxu Li, Xuandong Zhao, Hejia Geng, Xiaojun Wu, Junwei Zhou, Xiaokun Chen, Hanwen Xing, Yubo Li, Qunhong Zeng, Di Wang, Yuanli Wang, Roey Ben Chaim, Penghao Jiang, Haotian Shen, Luyang Kong, Xinyi Liu, Runhui Wang, Xuanqing Liu, Jiachen Li, Xin Lan, Yueqian Lin, Wengao Ye, Junwei He, Songlin Li, Yue Zhang, Yipeng Gao, Yijiang Li, Ze Ma, Liqiang Jing, Tianyu Wang, Kaixin Li, Yiqi Xue, Haoran Lyu, Yizhuo He, Yuchen Tian, Shutong Wu, Bowei Wang, Yixuan Gao, Bo Chen, Litong Liu, Sikai Cheng, Jiajun Bao, Shuaicheng Tong, Shuwen Xu, Terry Yue Zhuo, Tinghan Ye, Qi Qi, Miao Li, Longtai Liao, Zelin Tan, Chang Shi, Xilin Tang, Srinath Tankasala, Boqin Yuan, Yaoyao Qian, Jianhong Tu, Chenguang Wang, Yizhou Sun, Wei Wang, Aaron Taylor, Ziyue Yang, Changkun Guan, Zhikang Dong, Xinyu Zhang, Steven Dillmann, Han chung Lee, and Dawn Song, "Skills-Bench: Benchmarking How Well Agent Skills Work Across Diverse Tasks," 2026.

[5] Tingxu Han, Yi Zhang, Wei Song, Chunrong Fang, Zhenyu Chen, Youcheng Sun, and Lijie Hu, "SWE-Skills-Bench: Do Agent Skills Actually Help in Real-World Software Engineering?," 2026.

[6] Shanshan Zhong, Yi Lu, Jingjie Ning, Yibing Wan, Lihan Feng, Yuyi Ao, Leonardo F. R. Ribeiro, Markus Dreyer, Sean Ammirati, and Chenyan Xiong, "SkillLearnBench: Benchmarking Continual Learning Methods for Agent Skill Generation on Real-World Tasks," 2026.

[7] Zisu Huang, Jingwen Xu, Yifan Yang, Ziyang Gong, Qihao Yang, Muzhao Tian, Xiaohua Wang, Changze Lv, Xuemei Gao, Qi Dai, Bei Liu, Kai Qiu, Xue Yang, Dongdong Chen, Xiaoqing Zheng, and Chong Luo, "From Raw Experience to Skill Consumption: A Systematic Study of Model-Generated Agent Skills," 2026.